

Phân tích thuật toán

Bài giảng 04

PGS. TS. Nguyễn Thanh Bình

Khoa Toán - Tin học, Trường Đại học Khoa học Tự nhiên
Đại học Quốc gia TP. Hồ Chí Minh

Nội dung

- 1 Thuật toán đệ quy
- 2 Phương trình đệ quy
- 3 Giải phương trình đệ quy

Nội dung

- 1 Thuật toán đệ quy
- 2 Phương trình đệ quy
- 3 Giải phương trình đệ quy

Thuật toán đệ quy

- Là mở rộng cơ bản nhất của khái niệm thuật toán.
- Tư tưởng giải bài toán bằng đệ quy là đưa bài toán hiện tại về một bài toán *cùng loại, cùng tính chất* (đồng dạng) nhưng ở *cấp độ thấp hơn*, quá trình này tiếp tục cho đến khi bài toán được đưa về một cấp độ mà tại đó có thể giải được. Từ cấp độ này ta *lần ngược* để giải các bài toán ở cấp độ cao hơn cho đến khi giải xong bài toán ban đầu.

Thuật toán đệ quy

Ví dụ:

- Định nghĩa giai thừa: $n! = n * (n - 1)$ với $0! = 1$
- Dãy Fibonacci: $f_0 = 1, f_1 = 1$ và $f_n = f_{n-1} + f_{n-2}, \quad \forall n > 1$

Thuật toán đệ quy

Mọi thuật toán đệ quy gồm 02 phần:

- Phần cơ sở: Là các trường hợp không cần thực hiện lại thuật toán (không yêu cầu gọi đệ quy). Nếu thuật toán không có phần này thì sẽ bị lặp vô hạn và sinh lỗi khi thực hiện. Đôi lúc gọi là trường hợp dừng.
- Phần đệ quy: Là phần trong thuật toán có yêu cầu gọi đệ quy, yêu cầu thực hiện thuật toán ở một cấp độ thấp hơn.

Các loại đệ quy:

- Đệ quy đuôi.
- Đệ quy nhánh.
- Đệ quy tương hỗ.

Đệ quy đuôi

Đệ quy đuôi: Là loại đệ quy mà trong cấp đệ quy chỉ có duy nhất một lời gọi đệ quy xuống cấp thấp.

Ví dụ:

```
giaiThua(int n){  
    if (n == 0)  
        giaiThua = 1;  
    else giaiThua = n*giaiThua(n-1)  
}
```

Đệ quy đuôi

Ví dụ: Tìm kiếm nhị phân:

```
int searchBinary(int left, int right, int x){
    if (left < right){
        int mid = (left + right)/2;
        if (x == A[mid]) return mid;
        if (x < A[mid]) return searchBinary(left, mid - 1, x);
        return searchBinary(mid + 1, right, x);
    }
    return -1;
}
```


Đệ quy đuôi

Ví dụ: Phân tích ra thừa số nguyên tố (bài tập).

Đệ quy nhánh

Là dạng đệ quy mà trong quá trình đệ quy, lời gọi được thực hiện nhiều lần.

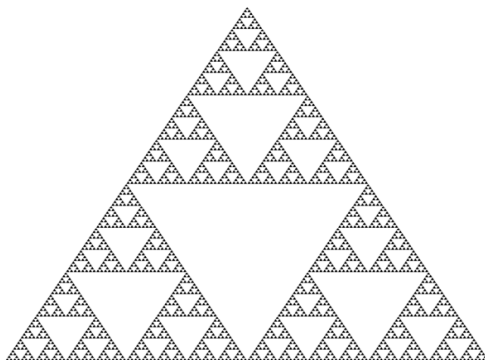
Ví dụ: Liệt kê tất cả hoán vị của n phần tử khác nhau.

Thuật toán

1. Xét tất cả các phần tử a_i với $i = 1 \dots n$.
2. Bỏ phần tử a_i ra khỏi dãy số.
3. Ghi nhận đã lấy ra phần tử a_i Hoán vị (dãy số).
4. Đưa phần tử a_i vào lại dãy số.
5. Nếu (dãy số) rỗng thì thứ tự các phần tử được lấy ra chính là một hoán vị.

Đệ quy tương hỗ

Là dạng đệ quy mà trong đó việc gọi có xoay vòng. Như A gọi B, B gọi C, C gọi A. Đây là trường hợp rất phức tạp.



Nội dung

- 1 Thuật toán đệ quy
- 2 Phương trình đệ quy
- 3 Giải phương trình đệ quy

Phương trình đệ quy

Phương trình đệ quy là một phương trình biểu diễn mối quan hệ giữa $T(n)$ và $T(k)$. Với $T(n)$ là **thời gian** thực hiện chương trình với kích thước dữ liệu nhập là n , $T(k)$ là **thời gian** thực hiện chương trình với kích thước dữ liệu nhập là k , $k < n$. Dựa trên chương trình đệ quy ta sẽ thành lập phương trình đệ quy.

Phương trình đệ quy

Dạng tổng quát của phương trình đệ quy:

$$T(n) = \begin{cases} C(n) \\ F(T(k)) + d(n) \end{cases}$$

Với:

- $C(n)$ là **thời gian** thực hiện chương trình ứng với trường hợp đệ quy dừng.
- $F(T(k))$ là hàm xác định thời gian theo $T(k)$.
- $d(n)$ là thừa số hằng.

Phương trình đệ quy

Ví dụ: phương trình đệ quy của bài toán giai thừa.

- Gọi $T(n)$ là **thời gian** tính n giai thừa thì $T(n-1)$ là **thời gian** tính $n-1$ giai thừa.
- Trong trường hợp $n = 0$ thì chỉ có 01 lệnh gán nên tốn $O(1) \rightarrow T(1) = C_1$
- Trong trường hợp $n > 0$, phải gọi đệ quy $giaiThua(n-1)$ nên tốn $T(n-1)$, sau khi có kết quả phải nhân kết quả với n và gán lại vào $giaiThua$. Thời gian để thực hiện phép nhân và gán là hằng C_2

Phương trình đệ quy

Vậy ta có:

$$T(n) = \begin{cases} C_1, & n = 0 \\ T(n-1) + C_2, & n > 0 \end{cases}$$

Ví dụ 2: Phương pháp MergeSort

- Chia dãy ban đầu thành 2 dãy gần bằng nhau.
- Chia đến khi nào chỉ còn một phần tử thì dừng chia
- Trộn các dãy lại thành dãy hoàn chỉnh được sắp xếp.

Phương trình đệ quy

Lý luận tương tự, ta có:

$$T(n) = \begin{cases} C_1, & n = 1 \\ 2T(\frac{n}{2}) + nC_2, & n > 1 \end{cases}$$

Nội dung

- 1 Thuật toán đệ quy
- 2 Phương trình đệ quy
- 3 Giải phương trình đệ quy

Giải phương trình đệ quy

Ta có những phương pháp sau:

- Phương pháp truy hồi.
- Đoán nghiệm.
- Lời giải tổng quát của một lớp các phương trình đệ quy.
- Phương pháp hàm sinh.

Phương pháp truy hồi

Thay thế các giá trị trong phương trình để suy ra $T(n)$.
Ví dụ: giải phương trình sau:

$$T(n) = \begin{cases} C_1, & n = 0 \\ T(n-1) + C_2, & n > 0 \end{cases}$$

Phương pháp truy hồi

Ta có:

$$\begin{aligned}T(n) &= T(n-1) + C_2 \\&= [T(n-2) + C_2] + C_2 = T(n-2) + 2C_2 \\&= [T(n-3) + C_2] + 2C_2 = T(n-3) + 3C_2 \\&\vdots \\&= T(n-i) + iC_2\end{aligned}$$

Quá trình kết thúc khi $n-i=0$ hay $i=n$. Khi đó

$$T(n) = T(0) + nC_2 = C_1 + nC_2 = O(n)$$

Phương pháp truy hồi

Ví dụ: giải phương trình sau:

$$T(n) = \begin{cases} C_1, & n = 1 \\ 2T(\frac{n}{2}) + nC_2, & n > 1 \end{cases}$$

Phương pháp truy hồi

Ta có:

$$\begin{aligned}T(n) &= 2T\left(\frac{n}{2}\right) + nC_2 \\&= 2\left[2T\left(\frac{n}{4}\right) + \frac{n}{2}C_2\right] + nC_2 = 4T\left(\frac{n}{4}\right) + 2nC_2 \\&= 4\left[2T\left(\frac{n}{8}\right) + \frac{n}{4}C_2\right] + 2nC_2 = 8T\left(\frac{n}{8}\right) + 3nC_2 \\&\vdots \\&= 2^i T\left(\frac{n}{2^i}\right) + inC_2\end{aligned}$$

Phương pháp truy hồi

Quá trình kết thúc khi $\frac{n}{2^i} = 1$ hay $i = \log(n)$.

Khi đó ta có:

$$\begin{aligned}T(n) &= nT(1) + nC_2\log(n) \\&= nC_1 + nC_2\log(n) \\&= O(n\log(n))\end{aligned}$$

Phương pháp truy hồi

Ví dụ: Giải các phương trình đệ quy sau với $T(1) = 1$:

1. $T(n) = 3T(\frac{n}{2}) + n$.
2. $T(n) = 4T(\frac{n}{3}) + n$, $T(0) = 1$, $T(1) = 1$.
3. $T(n) = T(\frac{n}{2}) + 1$
4. $T(n) = 2T(\frac{n}{2}) + \log(n)$
5. $T(n) = 2T(\frac{n}{2}) + n$

Phương pháp truy hồi

Đánh giá thuật toán sau:

```
procedure bugs(n)
  if == 1 then do something
  else
    bugs(n-1);
    bugs(n-2);
  for i := 1 to n do
    something
```

Phương pháp truy hồi

Đánh giá thuật toán sau:

```
procedure daffy(n)
  if n = 1 or n = 2 then do something
  else
    daffy(n-1);
    for i := 1 to n do
      do something new
    daffy(n-1);
```

Phương pháp truy hồi

Đánh giá thuật toán sau:

```
procedure multiply(y, z)
  // comment return the product yz
  if z = 0 then return(0) else
    if z is odd
      return multiply(2y, [z/2]) + y
    else
      return multiply(2y, [z/2])
```

Phương pháp truy hồi

Đánh giá thuật toán sau:

```
procedure elmer(n)
  if n = 1 then do something
  else if n = 2 then do something else
  else
    for i := 1 to n do
      elmer(n - 1);
    do something different
```

Phương pháp truy hồi

Đánh giá thuật toán sau:

```
procedure yosemite(n)
  if n = 1 then do something
  else
    for i := 1 to n - 1 do
      yosemite(i);
    do something completely different
```

Thank you for listening!